

Núcleo 2 Grafo de Conhecimento do Aluno

Datasets: **ASSISTments 2009** (acerto e tentativas) e **EdNet** (amostra, latência e navegação)

Aluna: Maria Clara Ribeiro Di Bragança

Fase 1 Business Understanding

Contexto e problema de negócio

A análise anterior (Núcleo 1, Text Mining) extraiu propriedades do texto de uma atividade (complexidade, intenção pedagógica e demanda psicomotora), mas o texto sozinho não descreve a criança que vai receber a atividade. O motor de adaptação precisa saber se essa criança já domina o conceito, e se ela costuma ter dificuldade em atividades visuais mas vai bem nas de áudio, por exemplo. Essa é a função do Grafo de Conhecimento: uma estrutura em que os nós são conceitos (e o aluno) e as arestas representam o domínio da criança sobre cada conceito, ponderadas por sinais comportamentais como taxa de acerto, latência e persistência.

Objetivo de negócio

Validar se é possível calibrar o peso inicial das arestas do grafo com dados públicos de comportamento real de alunos (ASSISTments e EdNet), antes de existir qualquer telemetria de uso real do sistema. Essa calibração offline depois é refinada de forma contínua pelos dados de uso real coletados em produção.

Objetivo da mineração de dados

Extrair de cada dataset os sinais comportamentais disponíveis, agregar por aluno e por conceito, combinar esses sinais em uma métrica única de domínio (o peso da aresta), verificar se essa métrica realmente antecipa o desempenho futuro do aluno e construir um grafo de exemplo com NetworkX para validar que a técnica produz um resultado interpretável.

Perguntas estratégicas de negócio

O ASSISTments oferece sinal de acerto e persistência (tentativas) suficiente para diferenciar conceitos dominados dos não dominados?

O EdNet oferece sinal de latência e abandono suficiente para o mesmo objetivo?

Uma métrica simples que combina esses sinais produz pesos pedagogicamente interpretáveis e que antecipam o desempenho futuro do aluno?

A relação de desempenho entre conceitos (aresta conceito com conceito) reflete uma dependência pedagógica ou só o nível geral do aluno?

Fase 2 Data Understanding

Importando as bibliotecas e olhando a estrutura de cada dataset antes de qualquer preparação.

```
In [1]: import glob
import warnings
warnings.filterwarnings("ignore")

import numpy as np
import pandas as pd
import networkx as nx

np.random.seed(42)
DATASETS = "../../datasets/raw"
```

2.1 ASSISTments 2009: acerto e persistência

O ASSISTments é um sistema tutor inteligente usado em escolas americanas. Cada linha do dataset é um aluno, e as colunas de interesse guardam uma lista com a sequência de interações dele:

user_id: identificador do aluno.

skill_names: lista com o nome do conceito (skill) de cada interação, por exemplo Circle Graph, Median ou Range.

grades: lista com '0' ou '1', indicando se o aluno acertou aquela interação.

attempt_counts: lista com o número de tentativas em cada interação (quanto maior, mais o aluno precisou tentar antes de prosseguir).

Como cada aluno tem listas, é preciso entender o tamanho dessas listas antes de transformar tudo em uma linha por interação.

```
In [2]: assistments = pd.read_parquet(f"{DATASETS}/assistments2009/assistments2009_train")
colunas_lista = ["skill_names", "grades", "attempt_counts"]

print("Dimensão do dataset (alunos x colunas):", assistments.shape)
print("Colunas:", list(assistments.columns))
print()

tamanhos = assistments[colunas_lista].apply(lambda coluna: coluna.apply(len))
print("Alunos cujas listas têm tamanhos diferentes entre as colunas:", int((tamanhos != tamanhos[0]).sum()))
print("Total de interações:", int(tamanhos["grades"].sum()))
```

```
print()
print("Interações por aluno:")
print(tamanhos["grades"].describe().round(2))
```

Dimensão do dataset (alunos x colunas): (4148, 6)

Colunas: ['user_id', 'skill_ids', 'skill_names', 'grades', 'attempt_counts', 'answer_types']

Alunos cujas listas têm tamanhos diferentes entre as colunas: 0

Total de interações: 274331

Interações por aluno:

```
count    4148.00
mean       66.14
std       128.27
min         1.00
25%         8.00
50%        20.00
75%        60.00
max       1040.00
```

Name: grades, dtype: float64

```
In [3]: print("Distribuição de grades (proporção):")
print(assistments["grades"].explode().value_counts(normalize=True).round(4))
print()

tentativas = assistments["attempt_counts"].explode().astype(float)
print("Distribuição de attempt_counts (proporção dos 8 valores mais frequentes):")
print(tentativas.value_counts(normalize=True).head(8).round(4))
print("Média:", round(tentativas.mean(), 3), "| máximo:", tentativas.max())
print()
skills = assistments["skill_names"].explode()
print("Skills distintas:", skills.nunique(), "| valores ausentes em skill_names:")
print("Skills mais frequentes:")
print(skills.value_counts().head(5))
```

Distribuição de grades (proporção):

```
grades
1    0.6616
0    0.3384
```

Name: proportion, dtype: float64

Distribuição de attempt_counts (proporção dos 8 valores mais frequentes):

```
attempt_counts
1.0    0.7904
2.0    0.0980
3.0    0.0380
0.0    0.0252
4.0    0.0200
5.0    0.0092
6.0    0.0050
7.0    0.0030
```

Name: proportion, dtype: float64

Média: 1.593 | máximo: 3824.0

```
Skills distintas: 101 | valores ausentes em skill_names: 0
Skills mais frequentes:
skill_names
Equation Solving Two or Fewer Steps      24253
Conversion of Fraction Decimals Percents 18742
Addition and Subtraction Integers        12741
Addition and Subtraction Fractions        11334
Equation Solving More Than Two Steps     8115
Name: count, dtype: int64
```

Interpretação

O dataset tem 4.148 alunos e 274.331 interações no total, e nenhum aluno tem listas de tamanhos diferentes entre as colunas (0 casos), ou seja, a explosão por posição na Fase 3 é segura, pois cada interação tem acerto e tentativa correspondentes. Também não existe valor ausente em skill_names nem em grades (0%), o que fica abaixo do limite de 1% da regra prática de dados faltantes e não é preocupante.

A distribuição de interações por aluno é bem assimétrica: a mediana é 20 interações, mas a média é 66,14 e o máximo chega a 1.040 (o desvio padrão de 128,27 é quase o dobro da média). Isso indica que existem poucos alunos com muita atividade puxando a média, e que a maioria dos alunos tem poucas interações (25% deles tem 8 ou menos), o que é algo que requer atenção porque qualquer taxa calculada por aluno e conceito vai ter poucos pontos. A proporção de acertos é de 66,16% contra 33,84% de erros, uma base desbalanceada mas sem ser extrema, e as 101 skills distintas são lideradas por Equation Solving Two or Fewer Steps (24.253 interações).

Sobre as tentativas, 79,04% das interações tem exatamente 1 tentativa e a média é 1,593, mas o máximo é 3.824, um valor que não faz sentido para uma única questão e que puxa a média para cima. A hipótese é erro de registro ou algum aluno que ficou tentando sem parar (não da para confirmar com o dicionário de dados). Existe também um valor 0 em 2,52% das interações que o dicionário não explica. Para a próxima etapa isso significa que a média de tentativas por aluno e skill é um sinal sensível a valores extremos, e que vai ser preciso filtrar combinações com poucas interações na preparação.

2.2 EdNet (amostra): latência e padrão de navegação

O EdNet é um log de interações de um aplicativo de preparação para a prova TOEIC (formato parecido com o de um app de idiomas). Cada aluno tem um arquivo CSV com uma sequência de eventos com timestamp em milissegundos:

enter: o aluno abre um bundle (item_id começa com 'b'), que é um conjunto de uma ou mais questões.

respond: o aluno responde uma questão (item_id começa com 'q').

submit: o aluno envia a resposta final do bundle.

quit: o aluno fecha uma tela. O significado exato desse evento precisa ser verificado nos dados antes de ser usado.

A **latência** (tempo até responder) é o sinal que o ASSISTments não tem, porque ele não registra timestamp. O catálogo de questões (questions.csv) informa a resposta correta e a coluna **part** (seção do exame, de 1 a 7), usada aqui como aproximação de conceito porque essa amostra pública não traz um dicionário que traduza os códigos de tag em nomes de conceito.

```
In [4]: questoes = pd.read_csv(f"{DATASETS}/ednet_sample/raw_data/ednet_content/question
arquivos_ednet = sorted(glob.glob(f"{DATASETS}/ednet_sample/raw_data/ednet_seque

print("Catálogo de questões:", questoes.shape)
resumo_parts = questoes.groupby("part").agg(
    n_questoes=("question_id", "count"),
    n_bundles=("bundle_id", "nunique"),
)
resumo_parts["questoes_por_bundle"] = (resumo_parts["n_questoes"] / resumo_parts
print(resumo_parts)
print()
print("Alunos (arquivos) disponíveis na amostra:", len(arquivos_ednet))

seq_exemplo = pd.read_csv(f"{DATASETS}/ednet_sample/raw_data/ednet_sequence/u276
print("Aluno de exemplo u2761:", seq_exemplo.shape, "| colunas:", list(seq_exemp
print(seq_exemplo["action_type"].value_counts())
```

Catálogo de questões:

```
(13169, 7)
      n_questoes  n_bundles  questoes_por_bundle
part
1             643         643                1.00
2            1662        1661                1.00
3            1266         422                3.00
4            1158         386                3.00
5            5703        5703                1.00
6            1335         334                4.00
7            1402         385                3.64
```

Alunos (arquivos) disponíveis na amostra: 4396

Aluno de exemplo u2761: (1475, 6) | colunas: ['timestamp', 'action_type', 'item_id', 'source', 'user_answer', 'platform']

action_type

enter 592

quit 316

respond 291

submit 276

Name: count, dtype: int64

Interpretação

O catálogo tem 13.169 questões e a amostra tem 4.396 alunos. A tabela por parte mostra uma diferença importante para a latência: as partes 1, 2 e 5 tem 1 questão por bundle, enquanto as partes 3 e 4 tem 3, a parte 6 tem 4 e a parte 7 tem 3,64 (em média). A parte 5 sozinha concentra 5.703 das questões (cerca de 43% do catálogo).

No aluno de exemplo (u2761) aparecem 1.475 eventos, sendo 592 enter, 316 quit, 291 respond e 276 submit. O número de enter é quase o dobro do de submit, o que indica que nem todo enter abre um bundle de questões, e isso precisa ser investigado (a hipótese é que o enter também é registrado ao abrir explicações e aulas). Para a próxima etapa, isso significa que só os enter com item_id de bundle podem ser usados para medir o tempo, e que nos bundles de várias questões o tempo desde o enter mistura o esforço de mais de uma pergunta.

```
In [5]: quits = seq_exemplo[seq_exemplo["action_type"] == "quit"]
print("Prefixo do item_id nos eventos quit do aluno u2761:")
print(quits["item_id"].str[0].value_counts())
```

```
Prefixo do item_id nos eventos quit do aluno u2761:
item_id
e      295
l       21
Name: count, dtype: int64
```

Interpretação

Dos 316 eventos quit do aluno u2761, 295 (93,4%) têm item_id começando com 'e' (explicação) e 21 (6,6%) começam com 'l' (aula), e nenhum é de bundle de questões. Ou seja, quit aqui significa que o aluno fechou uma tela de explicação ou de aula, e isso é uma ação normal, não um sinal de que ele desistiu da questão.

Isso é um lembrete importante de mineração de dados: a hipótese inicial de que quit seria abandono estava errada, e se ela tivesse sido usada direto, o peso das arestas ia ter um erro sistemático (todo aluno que lê explicações pareceria estar abandonando). Por isso o quit fica de fora da métrica, e o abandono precisa de outra definição, que é a do teste seguinte: bundle aberto sem submit correspondente.

```
In [6]: def taxa_abandono(caminho):
        seq = pd.read_csv(caminho)
        n_enter_bundle = ((seq["action_type"] == "enter") & (seq["item_id"].str.startswith('e')))
        n_submit = (seq["action_type"] == "submit").sum()
        return None if n_enter_bundle == 0 else 1 - (n_submit / n_enter_bundle)

taxas = [t for t in (taxa_abandono(f) for f in arquivos_ednet[:150]) if t is not None]
print("Alunos analisados:", len(taxas))
print("Média da taxa de abandono (bundle aberto sem submit):", round(sum(taxas) / len(taxas), 2))
print("Alunos com abandono maior que zero:", sum(1 for t in taxas if t > 0), "de", len(taxas))
```

```
Alunos analisados: 149
Média da taxa de abandono (bundle aberto sem submit): 0.0
Alunos com abandono maior que zero: 0 de 149
```

Interpretação

A taxa média de abandono, com a definição corrigida, é 0,000 e nenhum dos 149 alunos analisados tem abandono maior que zero (0 de 149). Todo bundle aberto tem um submit correspondente nesta amostra.

Isso indica que o sinal de abandono não é observável neste dataset público, provavelmente porque o aplicativo só grava a sessão quando ela é concluída (uma

hipótese, pois não há como confirmar só pelos dados). É uma limitação que já era esperada: sinais finos de persistência e abandono só vão existir a partir da telemetria real do sistema (eventos como `abandoned` e `hint_requested` do contrato `TelemetryEvent`), e por isso não dá para calibrar esse sinal na fase offline. A análise segue com acerto e latência, que são os sinais que o EdNet realmente oferece.

Fase 3 Data Preparation

Essa fase transforma as duas fontes em tabelas de uma linha por interação, agrega por aluno e conceito, e trata dois problemas encontrados na Fase 2: o evento `quit` não é abandono, e o abandono não é observável nesta amostra do EdNet.

ASSISTments: uma linha por interação e agregação por aluno e skill

O que eu altero: transformo as listas de cada aluno em uma linha por interação (explode), guardo a posição de cada interação na sequência do aluno, e depois agrupo por aluno e skill calculando taxa de acerto, número de interações e média de tentativas.

Por que preciso alterar: o peso da aresta é uma propriedade do par aluno e conceito, e não de uma interação isolada, então precisa ser calculado sobre a agregação.

Evidência da Fase 2: nenhuma lista tem tamanho diferente das outras, então a explosão alinhada por posição é segura (nenhuma interação fica sem acerto ou sem tentativa correspondente).

Risco de não fazer isso: usar uma interação isolada como se fosse domínio, o que daria 0 ou 1 e nunca um valor intermediário.

Como verifico: conferindo se o total de interações depois da explosão bate com o total de antes.

```
In [7]: interacoes = (
    assistments[["user_id"] + colunas_lista]
    .explode(colunas_lista)
    .rename(columns={"skill_names": "skill", "grades": "acerto", "attempt_counts": "tentativas"})
interacoes["acerto"] = interacoes["acerto"].astype(int)
interacoes["tentativas"] = interacoes["tentativas"].astype(float)
interacoes["posicao"] = interacoes.groupby("user_id").cumcount()

agregado_assistments = interacoes.groupby(["user_id", "skill"]).agg(
    taxa_acerto=("acerto", "mean"),
    n_interacoes=("acerto", "count"),
    tentativas_media=("tentativas", "mean"),
).reset_index()

print("Total de interações depois da explosão:", len(interacoes), "(antes:", int
print("Combinações aluno e skill:", len(agregado_assistments))
print()
```

```

print("Interações por combinação aluno e skill:")
print(agregado_assistments["n_interacoes"].describe().round(2))
for minimo in (1, 3, 5, 10):
    n = int((agregado_assistments["n_interacoes"] >= minimo).sum())
    print(f"    combinações com pelo menos {minimo:2d} interações: {n} ({n / len(a
agregado_assistments.head(6))

```

Total de interações depois da explosão: 274331 (antes: 274331)
 Combinações aluno e skill: 34968

Interações por combinação aluno e skill:

```

count    34968.00
mean         7.85
std        10.78
min         1.00
25%         2.00
50%         5.00
75%        10.00
max        290.00

```

Name: n_interacoes, dtype: float64

```

    combinações com pelo menos 1 interações: 34968 (100.0%)
    combinações com pelo menos 3 interações: 22897 (65.5%)
    combinações com pelo menos 5 interações: 18036 (51.6%)
    combinações com pelo menos 10 interações: 9117 (26.1%)

```

Out[7]:

	user_id	skill	taxa_acerto	n_interacoes	tentativas_media
0	14	Circle Graph	0.250000	12	0.750000
1	14	Median	0.000000	4	1.000000
2	14	Range	1.000000	3	1.000000
3	21825	Multiplication and Division Integers	1.000000	10	1.000000
4	21825	Table	0.571429	7	0.857143
5	51950	Equation Solving More Than Two Steps	1.000000	2	1.000000

Interpretação

Depois da explosão o total continua sendo 274.331 interações (igual ao de antes), então nenhuma foi perdida nem duplicada, e a agregação gerou 34.968 combinações de aluno e skill. O número de interações por combinação tem mediana 5 e média 7,85, mas 25% das combinações têm 2 interações ou menos e o máximo é 290.

Isso significa que metade das combinações tem poucas interações para uma taxa de acerto confiável (com 1 interação a taxa só pode ser 0 ou 1, nunca um valor intermediário). Exigir um mínimo tem um custo em cobertura: com pelo menos 3 interações ficam 65,5% das combinações, com 5 ficam 51,6% e com 10 ficam só 26,1%. Como o objetivo é um peso por conceito para cada criança, e uma criança nova vai ter poucas interações no começo, a solução não pode ser só filtrar (perderia justamente os casos de cold start), e por isso a Fase 4 compara uma formulação que suaviza a taxa quando há poucas interações.

O que eu altero: processo a sequência de cada aluno e calculo, para cada resposta, se ela foi correta (comparando com o catálogo) e duas versões de latência: a **latência do bundle** (tempo desde a abertura do bundle) e a **latência da questão** (tempo desde o último enter ou respond dentro do mesmo bundle). Só eventos enter de bundle (item_id com 'b') abrem um bundle, e o submit fecha.

Evidência da Fase 2: a tabela de questões por bundle mostra que algumas partes têm mais de uma questão por bundle.

Como verifico: comparando a latência média das duas versões por part.

```
In [8]: questoes_por_id = questoes.set_index("question_id")["part", "correct_answer"]

def processar_aluno_ednet(caminho):
    seq = pd.read_csv(caminho).sort_values("timestamp")
    registros, abertura_bundle, ultimo_evento = [], None, None
    for acao, item, resposta, momento in zip(seq["action_type"], seq["item_id"],
        if acao == "enter" and str(item).startswith("b"):
            abertura_bundle = ultimo_evento = momento
        elif acao == "respond" and abertura_bundle is not None and item in quest
            registros.append({
                "part": questoes_por_id.at[item, "part"],
                "acerto": int(resposta == questoes_por_id.at[item, "correct_answ
                "latencia_bundle_ms": momento - abertura_bundle,
                "latencia_questao_ms": momento - ultimo_evento,
            })
            ultimo_evento = momento
        elif acao == "submit":
            abertura_bundle = ultimo_evento = None
    return pd.DataFrame(registros)

N_ALUNOS_EDNET = 100
respostas = []
for i, caminho in enumerate(arquivos_ednet[:N_ALUNOS_EDNET]):
    r = processar_aluno_ednet(caminho)
    if len(r):
        r["aluno"] = i
        respostas.append(r)
respostas_ednet = pd.concat(respostas, ignore_index=True)

print("Alunos processados:", respostas_ednet["aluno"].nunique(), "| respostas:",
print("Respostas com latência da questão negativa:", int((respostas_ednet["laten
print()
comparacao_latencia = respostas_ednet.groupby("part").agg(
    n_respostas=("acerto", "count"),
```

```

    taxa_acerto=("acerto", "mean"),
    latencia_bundle_media_s=("latencia_bundle_ms", lambda s: s.mean() / 1000),
    latencia_questao_media_s=("latencia_questao_ms", lambda s: s.mean() / 1000),
)
comparacao_latencia.round(2)

```

Alunos processados: 98 | respostas: 59052
 Respostas com latência da questão negativa: 0

Out[8]:

	n_respostas	taxa_acerto	latencia_bundle_media_s	latencia_questao_media_s
part				
1	4659	0.70	20.53	17.56
2	7291	0.64	19.10	16.30
3	4057	0.65	64.99	29.45
4	3946	0.64	58.12	24.56
5	24645	0.53	21.12	15.78
6	10499	0.61	76.11	25.45
7	3955	0.59	177.55	52.95

Interpretação

Foram processados 98 dos 100 alunos (2 não tinham nenhuma resposta), com 59.052 respostas no total e nenhuma com latência negativa (0), o que confirma que os timestamps estão em ordem. Nas partes com 1 questão por bundle (1, 2 e 5) as duas latências são parecidas: 20,53 s contra 17,56 s na parte 1, 19,10 s contra 16,30 s na parte 2 e 21,12 s contra 15,78 s na parte 5. Já nas partes com várias questões por bundle a diferença é grande: 64,99 s contra 29,45 s na parte 3, 58,12 s contra 24,56 s na parte 4, 76,11 s contra 25,45 s na parte 6 e 177,55 s contra 52,95 s na parte 7 (mais de 3 vezes maior).

Isso confirma a hipótese levantada na Fase 2: a latência do bundle acumula o tempo das questões anteriores do mesmo bundle, e não representa o esforço de responder uma questão. Na prática, um aluno que responde 4 questões de 25 s cada apareceria com 100 s de latência na última, e seria lido como alguém com dificuldade quando o ritmo dele é normal. Por isso a latência da questão passa a ser a usada daqui em diante, e isso também muda a leitura de qualquer resultado que atribua latência alta ao esforço nas partes 3, 4, 6 e 7 (a parte 5, com mais respostas, 24.645 de 59.052, quase não é afetada).

```

In [9]: lat = respostas_ednet["latencia_questao_ms"] / 1000
print("Percentis da latência da questão (segundos):")
print(lat.quantile([0.25, 0.5, 0.75, 0.9, 0.95, 0.99]).round(1))
for teto in (30, 60, 120):
    print(f"Respostas acima de {teto} s: {(lat > teto).mean():.1%}")

```

Percentis da latência da questão (segundos):

0.25	7.7
0.50	14.8
0.75	23.9
0.90	42.4
0.95	64.3
0.99	137.4

Name: latencia_questao_ms, dtype: float64

Respostas acima de 30 s: 17.5%

Respostas acima de 60 s: 5.6%

Respostas acima de 120 s: 1.3%

Interpretação

A latência da questão tem mediana de 14,8 s e 75% das respostas ficam abaixo de 23,9 s, mas a cauda é longa: o percentil 90 é 42,4 s, o 95 é 64,3 s e o 99 é 137,4 s. Acima de 30 s ficam 17,5% das respostas, acima de 60 s ficam 5,6% e acima de 120 s ficam 1,3%.

Um teto de 60 segundos deixa de fora só 5,6% das respostas (as mais extremas, provavelmente de aluno que largou a questão aberta), e mantém a variação real dos outros 94,4%, então é uma escolha razoável. Esse teto só importa para as formulações que usam velocidade, que são testadas na Fase 4.

Conclusão da Fase 3

Deixei pronto para a Fase 4: a tabela de interações do ASSISTments agregada por aluno e skill (com a contagem de interações, que permite filtrar combinações pouco confiáveis), a tabela de respostas do EdNet com acerto e latência da questão (e não do bundle), e a decisão de tratar o quit como fechamento de explicação e o abandono como sinal não observável nesta amostra, ficando de fora da métrica de domínio.

Fase 4 Modeling

Esse notebook não tem uma variável alvo para treinar um classificador ou regressor (o objetivo é definir e validar uma métrica de domínio e construir um grafo), então os modelos supervisionados liberados nas aulas (Regressão Logística, Árvore de Decisão, Random Forest, LightGBM, XGBoost e SVC) não se aplicam aqui. O aprendizado supervisionado com esses modelos aparece no Núcleo 3, sobre o mesmo ASSISTments. Aqui a decisão é entre **formulações candidatas** para o peso da aresta, comparadas com o mesmo critério, e entre duas formas de construir as arestas entre conceitos.

4.1 Cada sinal sozinho separa domínio?

Antes de combinar sinais, vale testar se algum deles isolado já resolve, olhando a correlação entre os sinais de cada dataset. Filtrar combinações com poucas interações é importante porque uma taxa de acerto calculada com 1 interação só pode ser 0 ou 1.

```
In [10]: print("ASSISTments: correlação entre taxa de acerto e média de tentativas, por m
for minimo in (1, 3, 5, 10):
    filtrado = agregado_assistments[agregado_assistments["n_interacoes"] >= mini
    corr = filtrado["taxa_acerto"].corr(filtrado["tentativas_media"])
    print(f" mínimo {minimo:2d}: n = {len(filtrado):6d} | correlação = {corr:.3

print()
print("EdNet: latência da questão e acerto")
print(" correlação de Pearson:", round(respostas_ednet["latencia_questao_ms"].c
print(" correlação de Spearman:", round(respostas_ednet["latencia_questao_ms"].
print(" latência mediana (s) de quem errou e de quem acertou:")
print((respostas_ednet.groupby("acerto")["latencia_questao_ms"].median() / 1000)
```

ASSISTments: correlação entre taxa de acerto e média de tentativas, por mínimo de interações

mínimo 1:	n = 34968	correlação = -0.065
mínimo 3:	n = 22897	correlação = -0.106
mínimo 5:	n = 18036	correlação = -0.152
mínimo 10:	n = 9117	correlação = -0.119

EdNet: latência da questão e acerto

correlação de Pearson: -0.004

correlação de Spearman: 0.01

latência mediana (s) de quem errou e de quem acertou:

acerto

0 14.89

1 14.79

Name: latencia_questao_ms, dtype: float64

Interpretação

No ASSISTments a correlação entre taxa de acerto e média de tentativas é negativa e fraca em todos os cortes: -0,065 sem filtro (34.968 combinações), -0,106 com pelo menos 3 interações, -0,152 com pelo menos 5 e -0,119 com pelo menos 10. A direção é a esperada (quem tenta mais vezes tende a acertar menos), mas mesmo no melhor corte a correlação explica cerca de 2% da variação (0,152 ao quadrado), o que é muito pouco. Ela sobe até o corte de 5 interações, pois a taxa fica menos ruidosa, e cai no corte de 10 (9.117 combinações), o que pode ser porque sobram só os alunos e skills muito praticados, um grupo diferente do geral.

No EdNet a correlação entre latência da questão e acerto é praticamente nula (Pearson de -0,004 e Spearman de 0,010), e a latência mediana de quem errou (14,89 s) é quase igual à de quem acertou (14,79 s), uma diferença de 0,1 s. Isso faz sentido pedagogicamente, porque uma resposta rápida pode ser confiança ou pode ser um chute, e a latência sozinha não separa os dois casos.

Nenhum sinal isolado resolve o problema, mas a correlação entre dois sinais também não diz se um deles serve para antecipar o desempenho futuro do aluno, que é o que importa para o peso da aresta. Por isso o próximo passo testa exatamente isso.

4.2 Formulações candidatas para o peso da aresta

O teste que separa uma métrica útil de uma métrica só interpretável é saber se o peso calculado com o passado do aluno antecipa o desempenho dele no futuro. Para cada combinação com pelo menos 4 interações (e, em uma segunda rodada, com pelo menos 8), a primeira metade (em ordem de posição) calcula os sinais e o peso, e a segunda metade fornece a taxa de acerto futura. As formulações comparadas, todas entre 0 e 1 (1 é domínio total), são:

Só acerto: taxa de acerto do passado.

Só persistência (ASSISTments): $1 / (1 + \text{tentativas média})$. **Só velocidade** (EdNet): $1 - \min(\text{latência, teto}) / \text{teto}$, com teto de 60 segundos.

Composta: média simples do acerto com a persistência (ou com a velocidade), a ideia inicial do projeto por ser fácil de explicar ao professor.

Acerto suavizado: $(\text{acertos} + 1) / (\text{interações} + 2)$, que é o acerto puxado para 0,5 quando há poucas interações (com 1 acerto em 1 interação vale 0,67 e não 1,0), e converge para a taxa de acerto quando há muitas.

Todas usam o mesmo critério (correlação de Pearson e Spearman com a taxa de acerto futura). A premissa é que a ordem das listas é a ordem cronológica, o que não foi confirmado porque o ASSISTments 2009 usado aqui não traz timestamp. A escolha é entre 4 fórmulas sem parâmetros ajustados aos dados, então o risco de otimismo por seleção é pequeno, mas não zero.

```
In [11]: from scipy.stats import spearmanr

CAP_LATENCIA_MS = 60_000

def suavizar(taxa_acerto, n):
    return (taxa_acerto * n + 1) / (n + 2)

def montar_validacao(df, chaves, coluna_sinal, minimo, ordem=None):
    linhas = []
    for _, g in df.groupby(chaves):
        if len(g) < minimo:
            continue
        if ordem:
            g = g.sort_values(ordem)
        meio = len(g) // 2
        passado, futuro = g.iloc[:meio], g.iloc[meio:]
        linhas.append({"acerto": passado["acerto"].mean(), "n_passado": meio,
                      "sinal": passado[coluna_sinal].mean(), "acerto_futuro": f
        return pd.DataFrame(linhas)

def tabela_validade(df, colunas, nomes):
    return pd.DataFrame({
        "formulação": nomes,
        "Pearson": [df[c].corr(df["acerto_futuro"]) for c in colunas],
        "Spearman": [spearmanr(df[c], df["acerto_futuro"]).correlation for c in
    })

resultados_assist = {}
for minimo in (4, 8):
```

```

v = montar_validacao(interacoes, ["user_id", "skill"], "tentativas", minimo,
v["persistencia"] = 1 / (1 + v["sinal"])
v["composta"] = (v["acerto"] + v["persistencia"]) / 2
v["suavizado"] = suavizar(v["acerto"], v["n_passado"])
t = tabela_validade(v, ["acerto", "persistencia", "composta", "suavizado"],
["Só acerto", "Só persistência", "Composta (acerto + per
resultados_assist[minimo] = t
print(f"ASSISTments, mínimo de {minimo} interações: {len(v)} combinações alu
print(t.round(3).to_string(index=False))
print()

```

ASSISTments, mínimo de 4 interações: 20613 combinações aluno e skill | taxa de acerto futura média: 0.758

	formulação	Pearson	Spearman
	Só acerto	0.460	0.388
	Só persistência	0.093	0.163
Composta (acerto + persistência)		0.421	0.373
	Acerto suavizado	0.460	0.369

ASSISTments, mínimo de 8 interações: 11639 combinações aluno e skill | taxa de acerto futura média: 0.722

	formulação	Pearson	Spearman
	Só acerto	0.550	0.500
	Só persistência	0.112	0.216
Composta (acerto + persistência)		0.509	0.487
	Acerto suavizado	0.549	0.487

Interpretação

Com pelo menos 8 interações (11.639 combinações, taxa de acerto futura média de 0,722), o acerto do passado tem correlação de Pearson de 0,550 e de Spearman de 0,500 com o acerto futuro, ou seja, explica cerca de 30% da variação (0,55 ao quadrado). A persistência sozinha quase não tem sinal (Pearson de 0,112 e Spearman de 0,216), e a composta (média dos dois) fica pior que o acerto sozinho: 0,509 contra 0,550. O mesmo padrão aparece com pelo menos 4 interações (20.613 combinações): acerto 0,460, persistência 0,093, composta 0,421.

A hipótese para a composta piorar é que a persistência quase não varia (79% das interações têm 1 tentativa, então $1/(1+tentativas)$ fica perto de 0,5 para a maioria), e por isso a média com ela só diminui a diferença entre alunos bons e ruins, sem acrescentar informação. O acerto suavizado empata com o acerto no Pearson (0,549 contra 0,550 com pelo menos 8 interações, e 0,460 contra 0,460 com pelo menos 4) e fica um pouco abaixo no Spearman (0,487 contra 0,500, e 0,369 contra 0,388). Ou seja, no ASSISTments a suavização não traz ganho mensurável na correlação com o futuro.

A resposta para a primeira pergunta estratégica é que o ASSISTments oferece sinal suficiente de acerto, mas a persistência (tentativas) não tem sinal útil e piora a métrica quando entra na média. A decisão para a próxima etapa é usar o acerto suavizado como peso, pois ele empata com o acerto no ASSISTments, é melhor no EdNet (visto a seguir) e não gera pesos extremos (0 ou 1) com 1 ou 2 interações, que é o cenário do aluno novo e que este teste não consegue medir, porque exige pelo menos 4 interações no total.

```
In [12]: for minimo in (4, 8):
v = montar_validacao(respostas_ednet, ["aluno", "part"], "latencia_questao_m
v["velocidade"] = 1 - np.minimum(v["sinal"], CAP_LATENCIA_MS) / CAP_LATENCIA
v["composta"] = (v["acerto"] + v["velocidade"]) / 2
v["suavizado"] = suavizar(v["acerto"], v["n_passado"])
t = tabela_validade(v, ["acerto", "velocidade", "composta", "suavizado"],
["Só acerto", "Só velocidade", "Composta (acerto + veloc
print(f"EdNet, mínimo de {minimo} respostas: {len(v)} combinações aluno e pa
print(t.round(3).to_string(index=False))
print()
```

EdNet, mínimo de 4 respostas: 355 combinações aluno e part | taxa de acerto futur
a média: 0.584

	formulação	Pearson	Spearman
	Só acerto	0.311	0.359
	Só velocidade	-0.002	-0.014
Composta (acerto + velocidade)		0.188	0.196
	Acerto suavizado	0.356	0.371

EdNet, mínimo de 8 respostas: 314 combinações aluno e part | taxa de acerto futur
a média: 0.593

	formulação	Pearson	Spearman
	Só acerto	0.417	0.426
	Só velocidade	-0.008	-0.012
Composta (acerto + velocidade)		0.228	0.248
	Acerto suavizado	0.433	0.432

Interpretação

No EdNet, com pelo menos 8 respostas (314 combinações de aluno e parte), o acerto do passado tem Pearson de 0,417 e Spearman de 0,426 com o acerto futuro, a velocidade sozinha não tem sinal nenhum (-0,008 e -0,012) e a composta cai para 0,228 e 0,248, pouco mais da metade do valor do acerto sozinho. Com pelo menos 4 respostas (355 combinações) o padrão é o mesmo: 0,311 para o acerto, -0,002 para a velocidade e 0,188 para a composta.

Isso confirma, agora com o teste de desempenho futuro, o que a correlação da seção anterior já mostrava: a latência da questão não antecipa o acerto, e juntar ela ao acerto só adiciona ruído. Uma hipótese é que o tempo de resposta depende muito da própria questão (tamanho do texto, parte do exame) e pouco do aluno, e nesta análise a latência não foi normalizada pelo tempo típico da mesma questão, o que pode estar escondendo algum sinal real. O acerto suavizado é o melhor nas duas rodadas (Pearson de 0,433 contra 0,417 com pelo menos 8, e 0,356 contra 0,311 com pelo menos 4), o que é coerente com a ideia de que a suavização ajuda quando o histórico é curto. Como a amostra é pequena (98 alunos), uma diferença de 0,02 não é confiável, mas a distância entre a composta e o acerto (0,19 a 0,22 de diferença) é grande demais para ser acaso.

Para as perguntas estratégicas, o EdNet não oferece sinal de latência nem de abandono suficientes para entrar no peso da aresta, então o peso adotado é o acerto suavizado nas duas fontes.

4.3 Grafo aluno e conceito com NetworkX

Com a formulação escolhida (acerto suavizado, justificada pela comparação acima), o grafo tem um nó para o aluno, um nó para cada conceito e uma aresta entre eles com o peso de domínio. Cada aresta guarda também o número de interações e de acertos, para o peso poder ser atualizado depois sem reler o histórico. Em produção essa estrutura seria mantida para cada criança e atualizada por eventos de telemetria reais, no lugar dos datasets públicos.

```
In [13]: aluno_14 = agregado_assistments[agregado_assistments["user_id"] == 14].copy()
aluno_14["acertos"] = (aluno_14["taxa_acerto"] * aluno_14["n_interacoes"]).round(3)
aluno_14["peso_dominio"] = suavizar(aluno_14["taxa_acerto"], aluno_14["n_interacoes"])
print("Aluno 14 (ASSISTments):")
print(aluno_14[["skill", "n_interacoes", "taxa_acerto", "peso_dominio"]].round(3))

def construir_grafo_aluno(id_aluno, tabela_conceitos, coluna_conceito):
    grafo = nx.Graph()
    grafo.add_node(id_aluno, tipo="aluno")
    for _, linha in tabela_conceitos.iterrows():
        conceito = str(linha[coluna_conceito])
        grafo.add_node(conceito, tipo="conceito")
        grafo.add_edge(id_aluno, conceito, weight=float(linha["peso_dominio"]),
                       n_interacoes=int(linha["n_interacoes"]), acertos=int(linha["acertos"]))
    return grafo

grafo_aluno_14 = construir_grafo_aluno("aluno_14", aluno_14, "skill")
print()
print(f"Nós: {grafo_aluno_14.number_of_nodes()} | Arestas: {grafo_aluno_14.number_of_edges()}")
```

```
Aluno 14 (ASSISTments):
   skill  n_interacoes  taxa_acerto  peso_dominio
Circle Graph          12         0.25         0.286
      Median           4         0.00         0.167
      Range            3         1.00         0.800
```

Nós: 4 | Arestas: 3

Interpretação

No aluno 14 os três conceitos ficam com pesos diferentes: Range tem peso 0,800 (taxa de 100%, mas só 3 interações), Circle Graph tem 0,286 (taxa de 25% em 12 interações) e Median tem 0,167 (taxa de 0% em 4 interações). O grafo tem 4 nós (o aluno e 3 conceitos) e 3 arestas.

A suavização faz diferença aqui: sem ela Median teria peso 0 e Range teria peso 1, como se o sistema tivesse certeza absoluta com 3 ou 4 interações, e com ela o peso de Range (0,800) fica bem abaixo de 1. A ordem dos conceitos, do menor para o maior domínio, é Median, Circle Graph e Range, que é coerente com o que os dados mostram e pode ser explicada ao professor. Cada aresta guarda também o número de interações e de acertos, o que permite ao motor de adaptação saber o quão confiável é aquele peso, e permite atualizar sem reler o histórico.

4.4 Arestas entre conceitos

O grafo acima é bipartido (aluno e conceito): os conceitos não se conectam entre si. Um Grafo de Conhecimento completo também captura a relação entre os próprios conceitos, porque se o domínio de um tende a acompanhar o domínio de outro, isso indica proximidade cognitiva e orienta por onde começar uma intervenção.

A primeira ideia é a co-ocorrência (dois conceitos se conectam se muitos alunos interagem com os dois, medida pelo índice de Jaccard sobre o conjunto de alunos de cada conceito). A segunda é a **correlação de desempenho**: entre os alunos que interagiram com A e B, correlacionar a taxa de acerto em A com a taxa de acerto em B. Um cuidado é que um aluno forte tende a ir bem em tudo e um aluno fraco tende a ir mal em tudo, e isso sozinho já gera correlação entre conceitos sem nenhuma dependência pedagógica. Por isso também é calculada a correlação **centralizada por aluno**, em que a taxa de acerto de cada conceito é subtraída da média geral do próprio aluno, e as duas versões são comparadas.

```
In [14]: tabela_desempenho = interacoes.groupby(["user_id", "skill"])["acerto"].mean().reset_index()
pivot_desempenho = tabela_desempenho.pivot(index="user_id", columns="skill", values="acerto")

contagem_por_skill = pivot_desempenho.count()
conceitos_validos = contagem_por_skill[contagem_por_skill >= 20].index
pivot_filtrado = pivot_desempenho[conceitos_validos]
print("Conceitos com pelo menos 20 alunos:", len(conceitos_validos), "de", pivot_desempenho.columns.size)

presenca = pivot_filtrado.notna().astype(int).values
intersecao = presenca.T @ presenca
tamanho = presenca.sum(axis=0)
jaccard = intersecao / (tamanho[:, None] + tamanho[None, :] - intersecao)
jaccard_pares = jaccard[np.triu_indices_from(jaccard, k=1)]
print("Jaccard entre pares de conceitos: mediana =", round(float(np.median(jaccard_pares)), 3),
      "| percentil 90 =", round(float(np.quantile(jaccard_pares, 0.9)), 3))

mascara_superior = np.triu(np.ones((len(conceitos_validos),) * 2), k=1).astype(bool)
corr_bruta = pivot_filtrado.corr(min_periods=15).where(mascara_superior).stack()
corr_centralizada = pivot_filtrado.sub(pivot_filtrado.mean(axis=1), axis=0).corr()
print("Pares com correlação calculável:", len(corr_bruta))
print("Correlação média entre pares: bruta =", round(float(corr_bruta.mean()), 3))
print("Correlação entre as duas versões, par a par:", round(float(corr_centralizada.mean()), 3))

comparacao_limiar = pd.DataFrame({
    "arestas (correlação bruta)": [int((corr_bruta >= t).sum()) for t in (0.3, 0.4, 0.5, 0.6, 0.7)],
    "arestas (centralizada por aluno)": [int((corr_centralizada >= t).sum()) for t in (0.3, 0.4, 0.5, 0.6, 0.7)],
}, index=pd.Index([0.3, 0.4, 0.5, 0.6, 0.7], name="limiar"))
print(comparacao_limiar)

indice_conceito = {c: i for i, c in enumerate(conceitos_validos)}
top_pares = corr_centralizada.sort_values(ascending=False).head(6)
print("Pares com maior correlação centralizada e número de alunos em comum:")
print(pd.DataFrame({
    "par": [f"{a} | {c}" for a, c in top_pares.index],
    "correlação": top_pares.values.round(3),
    "alunos em comum": [int(intersecao[indice_conceito[a], indice_conceito[c]]) for a, c in top_pares.index]
}).to_string(index=False))
```

Conceitos com pelo menos 20 alunos: 87 de 101

Jaccard entre pares de conceitos: mediana = 0.183 | percentil 90 = 0.399

Pares com correlação calculável: 3198

Correlação média entre pares: bruta = 0.194 | centralizada = -0.033

Correlação entre as duas versões, par a par: 0.637

	arestas (correlação bruta)	arestas (centralizada por aluno)
limiar		
0.3	864	86
0.4	310	27
0.5	75	8
0.6	14	3
0.7	2	2

Pares com maior correlação centralizada e número de alunos em comum:

	alunos em comum	par	correla
721	45	Angles on Parallel Lines Cut by a Transversal Nets of 3D Figures	0.
720	15	Angles on Parallel Lines Cut by a Transversal Rounding	0.
605	28	Midpoint Stem and Leaf Plot	0.
573	16	Divisibility Rules Greatest Common Factor	0.
567	57	Calculations with Similar Figures Greatest Common Factor	0.
541	31	Angles on Parallel Lines Cut by a Transversal Interior Angles Triangle	0.

Interpretação

Dos 101 conceitos, 87 têm pelo menos 20 alunos e entram na análise (14 conceitos, ou 13,9%, ficam de fora por terem base pequena). A co-ocorrência pelo Jaccard tem mediana de 0,183 e percentil 90 de 0,399, ou seja, o par típico de conceitos divide só uns 18% dos alunos da união dos dois. A hipótese é que boa parte dos alunos passa pelo mesmo currículo, e por isso a co-ocorrência mede mais o currículo comum do que uma relação entre os conceitos (e ela também não diz nada sobre o desempenho). Por isso a correlação de desempenho é usada no lugar dela, com 3.198 pares calculáveis (85% dos 3.741 possíveis).

A correlação bruta tem média de 0,194, mas a centralizada por aluno (que remove o nível geral de cada aluno) tem média de -0,033, e a correlação entre as duas versões, par a par, é de 0,637. O efeito aparece com clareza nas arestas: com limiar de 0,5 são 75 arestas na versão bruta e só 8 na centralizada (queda de cerca de 89%), com 0,4 são 310 contra 27, e com 0,3 são 864 contra 86. Isso indica que a maior parte da correlação entre conceitos vem do nível geral do aluno (quem vai bem em um tende a ir bem em todos), e não de uma dependência entre os conceitos.

Os pares de maior correlação centralizada misturam casos coerentes e casos estranhos: Divisibility Rules com Greatest Common Factor (0,573) e Angles on Parallel Lines com Interior Angles Triangle (0,541) fazem sentido matemático, mas Angles on Parallel Lines com Rounding (0,720, só 15 alunos em comum) e Midpoint com Stem and Leaf Plot (0,605, 28 alunos) não têm relação óbvia, e a hipótese é que sejam ruído de amostra pequena, já que o mínimo exigido foi de 15 alunos. Por isso o grafo entre conceitos usa a

versão centralizada, com limiar de 0,3 (86 arestas, uma quantidade parecida com as 75 da versão bruta em 0,5, o que permite comparar).

```
In [15]: LIMIAR_CORRELACAO = 0.3

grafo_conceitos = nx.Graph()
grafo_conceitos.add_nodes_from(conceitos_validos)
for (a, c), valor in corr_centralizada.items():
    if valor >= LIMIAR_CORRELACAO:
        grafo_conceitos.add_edge(a, c, weight=round(float(valor), 3))

componentes = sorted(nx.connected_components(grafo_conceitos), key=len, reverse=True)
print(f"Grafo conceito com conceito (correlação centralizada, limiar {LIMIAR_CORRELACAO}):")
print(f" {grafo_conceitos.number_of_nodes()} nós | {grafo_conceitos.number_of_edges()} arestas | {len(componentes)} componentes | maior componente = {len(componentes[0])} nós | isolados = {sum(1 for n in grafo_conceitos.nodes if n not in component[0] for component in componentes)}")
print(f" grau médio: {np.mean([d for _, d in grafo_conceitos.degree()]):.2f} | densidade: {grafo_conceitos.number_of_edges()/(grafo_conceitos.number_of_nodes()*(grafo_conceitos.number_of_nodes()-1)):.2f}")

grau = dict(grafo_conceitos.degree())
centralidade = nx.betweenness_centrality(grafo_conceitos)
tabela_centralidade = pd.DataFrame({"grau": grau, "intermediacao": centralidade})
print()
print("Top 8 conceitos por centralidade de intermediação (com o grau de cada um)")
print(tabela_centralidade.head(8).round(4))
```

Grafo conceito com conceito (correlação centralizada, limiar 0.3):

87 nós | 86 arestas | 21 componentes | maior componente = 63 nós | isolados = 19

grau médio: 1.98 | densidade: 0.023

Top 8 conceitos por centralidade de intermediação (com o grau de cada um):

	grau	intermediacao
Nets of 3D Figures	10	0.1884
Midpoint	9	0.1675
Rotations	8	0.1179
Multiplication and Division Positive Decimals	3	0.1025
Absolute Value	6	0.0944
Interior Angles Figures with More than 3 Sides	3	0.0841
Ordering Integers	5	0.0686
Area Parallelogram	6	0.0590

Interpretação

O grafo entre conceitos tem 87 nós e 86 arestas, organizados em 21 componentes, sendo que a maior tem 63 nós e 19 conceitos ficam isolados (21,8%, sem nenhum par acima do limiar). O grau médio é de 1,98 e a densidade de 0,023, então é um grafo bem esparso. Nem todo conceito tem relação forte de desempenho com outro, e o grafo reflete isso em vez de forçar conexões.

Os conceitos mais centrais pela intermediação são Nets of 3D Figures (0,1884, grau 10), Midpoint (0,1675, grau 9) e Rotations (0,1179, grau 8), todos de geometria, seguidos de Multiplication and Division Positive Decimals (0,1025), que tem grau só 3 mas funciona como ponte entre grupos. A hipótese é que os conceitos de geometria são estudados na mesma unidade e por isso o desempenho deles anda junto, mas isso é correlação e não prova que um seja pré-requisito do outro. Para o objetivo do projeto isso indica

candidatos a serem verificados com prioridade, e não uma ordem de ensino, e a próxima etapa testa se esses conceitos centrais se mantêm quando o limiar muda.

```
In [16]: origem, destino = "Median", "Circle Graph"
existe_caminho = nx.has_path(grafo_conceitos, origem, destino)
print(f"Existe caminho entre {origem} e {destino} no grafo centralizado:", existe_caminho)
if existe_caminho:
    caminho = nx.shortest_path(grafo_conceitos, origem, destino)
    print("Distância:", len(caminho) - 1, "| caminho:", " -> ".join(caminho))
print("Grau de cada um no grafo:", grafo_conceitos.degree(origem), "e", grafo_conceitos.degree(destino))
print("Correlação centralizada entre os dois:", round(float(corr_centralizada.get((origem, destino), 0)), 3))
print("Correlação bruta entre os dois:", round(float(corr_bruta.get((origem, destino), 0)), 3))

print()
print("Perfil do aluno 14, do menor para o maior peso:")
print(aluno_14[["skill", "peso_dominio"]].sort_values("peso_dominio").to_string())
```

```
Existe caminho entre Median e Circle Graph no grafo centralizado: False
Grau de cada um no grafo: 0 e 3
Correlação centralizada entre os dois: 0.08
Correlação bruta entre os dois: 0.456
```

```
Perfil do aluno 14, do menor para o maior peso:
      skill  peso_dominio
Median      0.167
Circle Graph 0.286
Range       0.800
```

Interpretação

Os dois conceitos mais fracos do aluno 14, Median (peso 0,167) e Circle Graph (peso 0,286), estão desconectados no grafo centralizado: Median tem grau 0 (não passa do limiar de 0,3 com nenhum outro conceito) e Circle Graph tem grau 3, mas nenhuma dessas conexões leva a Median, ou seja, não existe caminho entre os dois. A correlação centralizada entre os dois é de apenas 0,080, contra 0,456 na versão bruta.

Isso muda a conclusão que o notebook tinha antes: a correlação bruta de 0,456 sugeria que a dificuldade do aluno nos dois conceitos era uma demanda cognitiva compartilhada, mas depois de remover o nível geral dos alunos a correlação some (0,080), o que indica que ela vinha de alunos que vão mal em tudo, e não de uma relação específica entre Median e Circle Graph. Para o aluno 14, então, o grafo não indica um caminho de menor atrito entre os dois, e a adaptação deve tratar cada conceito separadamente, sem assumir que reforçar um ajuda o outro.

Fase 5 Evaluation

As decisões da Fase 4 dependem de dois números escolhidos sem validação externa: o limiar de correlação do grafo entre conceitos e o critério de conceitos com pelo menos 20 alunos. A avaliação abaixo verifica se a estrutura do grafo (e os conceitos centrais) é estável quando o limiar muda, porque um grafo que muda por completo com uma pequena variação de limiar não serve de base para decidir o que ensinar primeiro.

```
In [17]: def top5_intermediacao(grafo):
          c = nx.betweenness_centrality(grafo)
          return set(sorted(c, key=c.get, reverse=True)[:5])

referencia = top5_intermediacao(grafo_conceitos)
linhas = []
for limiar in (0.25, 0.30, 0.35, 0.40, 0.45):
    g = nx.Graph()
    g.add_nodes_from(conceitos_validos)
    g.add_edges_from(corr_centralizada[corr_centralizada >= limiar].index)
    comps = sorted(nx.connected_components(g), key=len, reverse=True)
    linhas.append({
        "limiar": limiar, "arestas": g.number_of_edges(), "componentes": len(comps),
        "maior componente": len(comps[0]), "isolados": sum(1 for n in g if g.degree[n] == 0),
        "conceitos em comum no top 5 (vs 0,30)": len(set(referencia) & set(top5_intermediacao(g)))
    })
pd.DataFrame(linhas)
```

```
Out[17]:
```

	limiar	arestas	componentes	maior componente	isolados	conceitos em comum no top 5 (vs 0,30)
0	0.25	131	14	74	13	2
1	0.30	86	21	63	19	5
2	0.35	49	41	33	34	3
3	0.40	27	60	15	54	2
4	0.45	13	74	6	68	3

Interpretação

Variar o limiar em passos de 0,05 muda muito o grafo: com 0,25 são 131 arestas e 14 componentes, com 0,30 são 86 arestas e 21 componentes, com 0,35 são 49 arestas e 41 componentes, e com 0,45 são só 13 arestas, com 68 conceitos isolados. Os conceitos centrais também não se mantêm: dos 5 mais centrais em 0,30, só 2 continuam entre os 5 mais centrais em 0,25, 3 em 0,35, 2 em 0,40 e 3 em 0,45.

Isso indica que a estrutura do grafo entre conceitos é sensível à escolha do limiar, e que a lista de conceitos centrais não é robusta com os dados disponíveis (a variação de 0,05 troca até 3 dos 5 conceitos). O limiar de 0,3 foi escolhido por comparação de densidade com a versão bruta, e não por validação externa. Isso limita o uso do grafo: não é seguro basear uma decisão automática de sequenciamento nele, e ele deve ser tratado como exploratório até existir uma referência independente (como um mapa de pré-requisitos definido por professores) para validar as arestas.

Fase 6 Deployment

O grafo construído usa o histórico completo de um aluno, mas um aluno novo não tem histórico nenhum. Esse problema é conhecido como **cold start**: o grafo nasce com um sinal grosseiro e é refinado por eventos de uso real. A função abaixo define o contrato de

atualização incremental do peso, coerente com a formulação escolhida na Fase 4: cada evento soma uma interação e um acerto (ou não), e o peso é recalculado como acerto suavizado. Assim uma aresta nova nasce perto de 0,5 (incerta) e não em 0 ou 1, e só se afasta desse valor conforme as evidências se acumulam.

```
In [18]: def atualizar_aresta_grafo(grafo, id_aluno, conceito, acertou):
    if not grafo.has_edge(id_aluno, conceito):
        grafo.add_node(conceito, tipo="conceito")
        grafo.add_edge(id_aluno, conceito, weight=0.5, n_interacoes=0, acertos=0)
    aresta = grafo[id_aluno][conceito]
    aresta["n_interacoes"] += 1
    aresta["acertos"] += int(acertou)
    aresta["weight"] = round((aresta["acertos"] + 1) / (aresta["n_interacoes"] + 1))
    return grafo

aresta = grafo_aluno_14["aluno_14"]["Median"]
historico_pesos = [aresta["weight"]]
for _ in range(6):
    atualizar_aresta_grafo(grafo_aluno_14, "aluno_14", "Median", acertou=True)
    historico_pesos.append(aresta["weight"])
print("Peso de Median a cada acerto seguido:", historico_pesos)

atualizar_aresta_grafo(grafo_aluno_14, "aluno_14", "Median", acertou=False)
print("Depois de 1 erro:", aresta["weight"], "| interações:", aresta["n_interacoes"])

atualizar_aresta_grafo(grafo_aluno_14, "aluno_14", "Fractions", acertou=True)
print("Conceito novo (cold start), depois de 1 acerto:", grafo_aluno_14["aluno_14"]
```

Peso de Median a cada acerto seguido: [0.167, 0.286, 0.375, 0.444, 0.5, 0.545, 0.583]

Depois de 1 erro: 0.538 | interações: 11 | acertos: 6

Conceito novo (cold start), depois de 1 acerto: 0.667

Interpretação

Partindo de Median com peso 0,167 (0 acertos em 4 interações), 6 acertos seguidos elevam o peso para 0,286, 0,375, 0,444, 0,500, 0,545 e 0,583, e um erro depois disso reduz para 0,538 (6 acertos em 11 interações). Um conceito novo (cold start) nasce com peso 0,667 depois de 1 acerto, e nasceria com 0,333 depois de 1 erro, e nunca em 0 ou 1.

O comportamento é estável, porque nenhum evento sozinho muda muito o peso (o erro custa 0,045), o que evita que o grafo oscile. Mas o custo é a lentidão: depois de 6 acertos seguidos o peso ainda está pouco acima do valor neutro de 0,5, porque o histórico ruim (4 erros) continua pesando igual ao mais recente. Isso é uma limitação, pois uma criança aprende e o desempenho de 3 semanas atrás vale menos que o de hoje. O peso por recência (dar mais importância às interações recentes) não foi testado, porque o ASSISTments não tem timestamp, e só a telemetria real do sistema vai permitir calibrar isso.

Conclusão e recomendação

O que os dados permitiram concluir, pergunta por pergunta:

1. O ASSISTments oferece sinal de acerto e persistência suficiente? Só o acerto. O acerto do passado tem correlação de 0,550 com o acerto futuro (explica cerca de 30% da variação), enquanto a persistência (tentativas) tem apenas 0,112 e piora a métrica quando entra na média (composta 0,509).

2. O EdNet oferece sinal de latência e abandono? Não. A latência da questão tem correlação de -0,004 com o acerto (e velocidade de -0,008 com o acerto futuro), e o abandono não é observável (0 de 149 alunos). Além disso, foram encontrados e corrigidos dois problemas de interpretação: o evento quit é fechamento de explicação (93,4% dos casos) e não abandono, e a latência do bundle chega a mais de 3 vezes a latência da questão nas partes com vários itens.

3. Uma métrica simples que combina os sinais é interpretável e antecipa o desempenho futuro? A combinação inicial (acerto com persistência ou velocidade) não antecipa melhor que o acerto sozinho (0,509 contra 0,550 no ASSISTments e 0,228 contra 0,417 no EdNet). Após comparar as formulações, o peso adotado é o acerto suavizado (0,549 e 0,433), que é interpretável (0 a 1) e evita pesos extremos com poucas interações.

4. A relação entre conceitos reflete dependência ou o nível geral do aluno? Em grande parte o nível geral: com limiar de 0,5 são 75 arestas na correlação bruta e só 8 na centralizada. O grafo centralizado é esparsos (86 arestas, 19 conceitos isolados) e instável ao limiar (os 5 conceitos mais centrais trocam de 2 a 3 conforme o limiar muda).

Recomendação do que fazer no sistema agora:

1. Usar o acerto suavizado como peso inicial das arestas aluno e conceito, nascendo em 0,5 no cold start, porque é o único sinal com evidência de antecipar o desempenho futuro. Persistência, latência e número de interações podem ser guardados como atributos da aresta, mas ficam fora do cálculo do peso até que a telemetria real mostre que eles agregam.

2. Não usar o grafo entre conceitos para decidir sequência de ensino de forma automática. Ele serve como exploração (quais conceitos investigar), mas as relações de pré-requisito precisam vir de uma fonte independente, como um mapa definido pelos professores.

3. Tratar o peso como estimativa com incerteza: mostrar sempre junto o número de interações, e evitar decisões fortes quando ele for baixo.

O que falta em dados para chegar em um resultado melhor:

Timestamp por interação: o ASSISTments 2009 não tem, então a ordem cronológica é uma premissa não confirmada, e não dá para pesar a recência (o peso atual demora a refletir aprendizado recente).

Latência normalizada pela questão: comparar o tempo do aluno com o tempo típico daquela mesma questão, para separar o efeito do aluno do efeito da questão.

Eventos de abandono e pedido de dica: não existem nos datasets públicos, e o contrato TelemetryEvent do sistema (abandoned, hint_requested) é o que vai fornecer

esses sinais.

Mais alunos por par de conceitos: as correlações entre conceitos usaram no mínimo 15 alunos em comum, e alguns dos pares mais fortes tinham só 15 a 28, o que é pouco para separar relação real de ruído.

Um mapa de pré-requisitos definido por professores: para validar (ou substituir) as arestas entre conceitos.

Dados de crianças neurodivergentes: os dois datasets são de alunos em geral (escolas americanas e um app de preparação para o TOEIC), então estes números validam a técnica, e não o desempenho esperado com as crianças do CogniKids (TEA, TDAH e Dislexia), que só a telemetria do próprio sistema vai mostrar.

ESSE NOTEBOOK FOI FEITO COM IA PARA O NOSSO APRENDIZADO E CONFORTO MENTAL, ASSIM VAI FICAR CLARO QUE NÃO FOMOS NÓS QUE FIZEMOS SEM ATRAPALHAR O APRENDIZADO DELAS.